

Introduction to Deep Reinforcement Learning

Maheed Hatem Ahmed

Research Assistant, ECE, Purdue University

ahmed237@purdue.edu

3/13/2024



Agenda

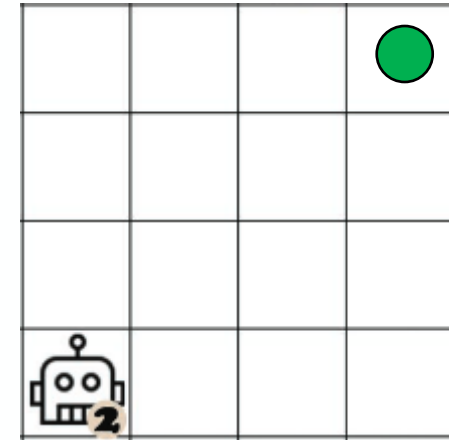
- Why Deep Reinforcement Learning?
- DQN Algorithm
- Taxonomy of Deep Reinforcement Learning Algorithms
- Gymnasium Environments
- DQN Code Demo
- Policy Gradient algorithms

Agenda

- Why Deep Reinforcement Learning?
- DQN Algorithm
- Taxonomy of Deep Reinforcement Learning Algorithms
- Gymnasium Environments
- DQN Code Demo
- Policy Gradient algorithms

Markov Decision Process

- An MDP is defined as the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, p_0)$
 - $\mathcal{S}$ is the set representing the state space
 - $\mathcal{A}$ is the set representing the action space
 - $P: \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the transition probability
 - $r: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function
 - $\gamma \in [0,1]$ is the discount factor
 - p_0 is the probability distribution for the initial state
- A policy $\pi: \mathcal{S} \rightarrow \Delta(\mathcal{A})$ is trained to map any given state to an optimal action.



Reinforcement Learning Algorithms Examples

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$
2. Policy Evaluation
Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s',r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
 policy-stable $\leftarrow$ true
 For each $s \in \mathcal{S}$:
 old-action $\leftarrow \pi(s)$
 $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$
 If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false
 If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:
 $\Delta \leftarrow 0$
 Loop for each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
 $\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$

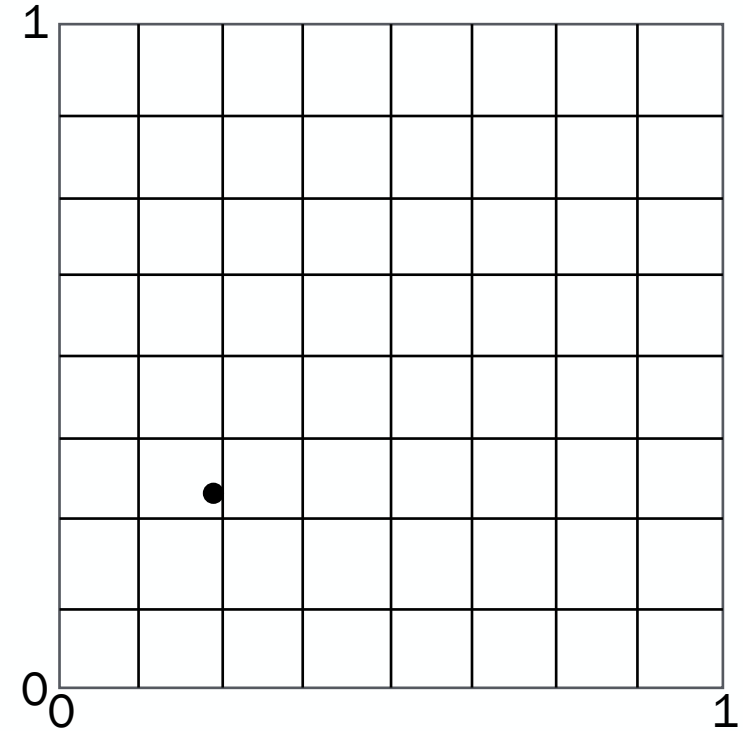
Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:
 Initialize S
 Loop for each step of episode:
 Choose A from S using policy derived from Q (e.g., ε -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$
 until S is terminal

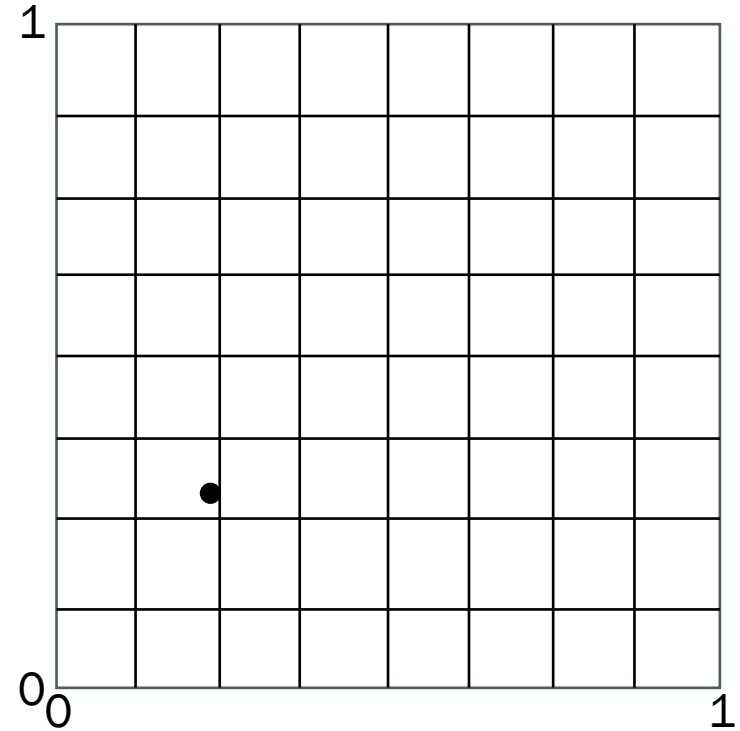
Markov Decision Processes with Continuous State or Action Space

- The value function, Q-function, and the advantage function are represented as tables for MDPs with discrete state spaces.
- In the case of a continuous state space, how to represent the functions and policy?
- One solution is to discretize the state space.
 - Curse of Dimensionality
 - Loss of information
 - Grid resolution
 - Worse generalization

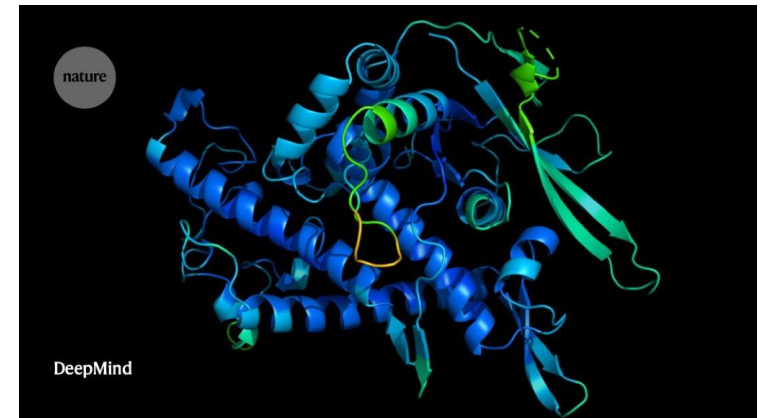
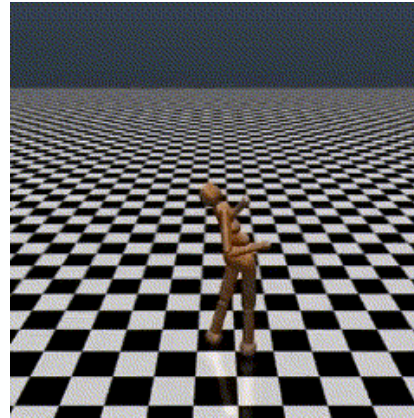
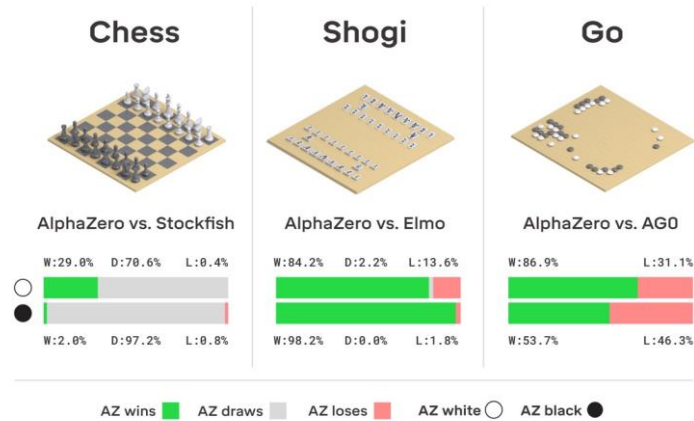


Markov Decision Processes with Continuous State or Action Space

- Instead, a function approximator $f_\theta: \mathcal{S} \rightarrow \mathbb{R}$ or $f_\theta: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is parameterized by θ can be trained to approximate the functions.
- Similarly, a function approximator $\pi_\phi: \mathcal{S} \rightarrow \Delta(\mathcal{A})$ parameterized by ϕ can be trained to approximate a policy with a potentially continuous action space.



Recent Success of Deep Reinforcement Learning



Agenda

- Why Deep Reinforcement Learning?
- DQN Algorithm
- Taxonomy of Deep Reinforcement Learning Algorithms
- Gymnasium Environments
- DQN Code Demo
- Policy Gradient algorithms

Q-Learning

- The optimal action-value function

$$Q^*(s, a) = \max_{\pi} \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t), a_t \sim \pi(s_t)} [\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | s_0 = s, a_0 = a, \pi]$$

obeys the bellman optimality condition

$$Q^*(s, a) = r(s, a) + \gamma \max_{a'} \mathbb{E}_{s' \sim P(\cdot | s)} [Q^*(s', a')].$$

- In Q-learning the basic idea is to collect a set of transitions (s, a, s') using some behavioral policy π_b and update the Q-function with each iteration as

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Deep Q-Learning

- Learning a table-based Q-value for each state action pair might be infeasible when the state space is huge or continuous.
- Instead, a function approximator Q_θ parameterized by θ is trained such that $Q_\theta(s, a) \approx Q^*(s, a)$.
- A neural network can be used as a function approximator for the Q-function and is referred to as the Q-network.
- The loss function at iteration i is defined as

$$\mathcal{L}(\theta_i) = \mathbb{E}_{s, a \sim \pi_b, s' \sim P(\cdot|s)} \left[\left(r(s, a) + \gamma \max_{a'} Q_{\theta_{i-1}}(s', a') - Q_{\theta_i}(s, a) \right)^2 \right]$$

- Given a batch of samples (s_k, a_k, s'_k) of size B , the expectation can be estimated as

$$\mathcal{L}(\theta_i) = \frac{1}{B} \sum_{k=1}^B \left(r(s_k, a_k) + \gamma \max_{a'_k} Q_{\theta_{i-1}}(s'_k, a'_k) - Q_{\theta_i}(s, a) \right)^2$$

Human-level control through deep reinforcement learning

- In this paper, deep Q-learning is used to train an agent to achieve human-level play of Atari games using the pixels as states.
- The state at a given timestep is represented by the most recent preprocessed 4 frames stacked together.
- A convolutional neural network is used as the function approximator.



Human-level control through deep reinforcement learning

- Experience replay
 - If recent transitions are only used to update the parameters of the network, the network might forget transitioned experienced earlier in the training.
 - Hence, a replay buffer holds experience up to N previous steps.
 - Batches of size B are sampled from the replay buffer during training.
- Target network
 - Instead of using the constantly changing Q-network to compute the targets
$$r(s, a) + \gamma \max_{a'} Q_{\theta_{i-1}}(s', a')$$
a separate target network $Q_{\theta_{-i}}$ with parameters θ_{-i} is used.
 - The target network parameters are updated every C steps to match the Q-network, i.e.,
$$\theta_{-i} \leftarrow \theta_i$$

Human-level control through deep reinforcement learning

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

 With probability ε select a random action a_t

 otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

 Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

 Every C steps reset $\hat{Q} = Q$

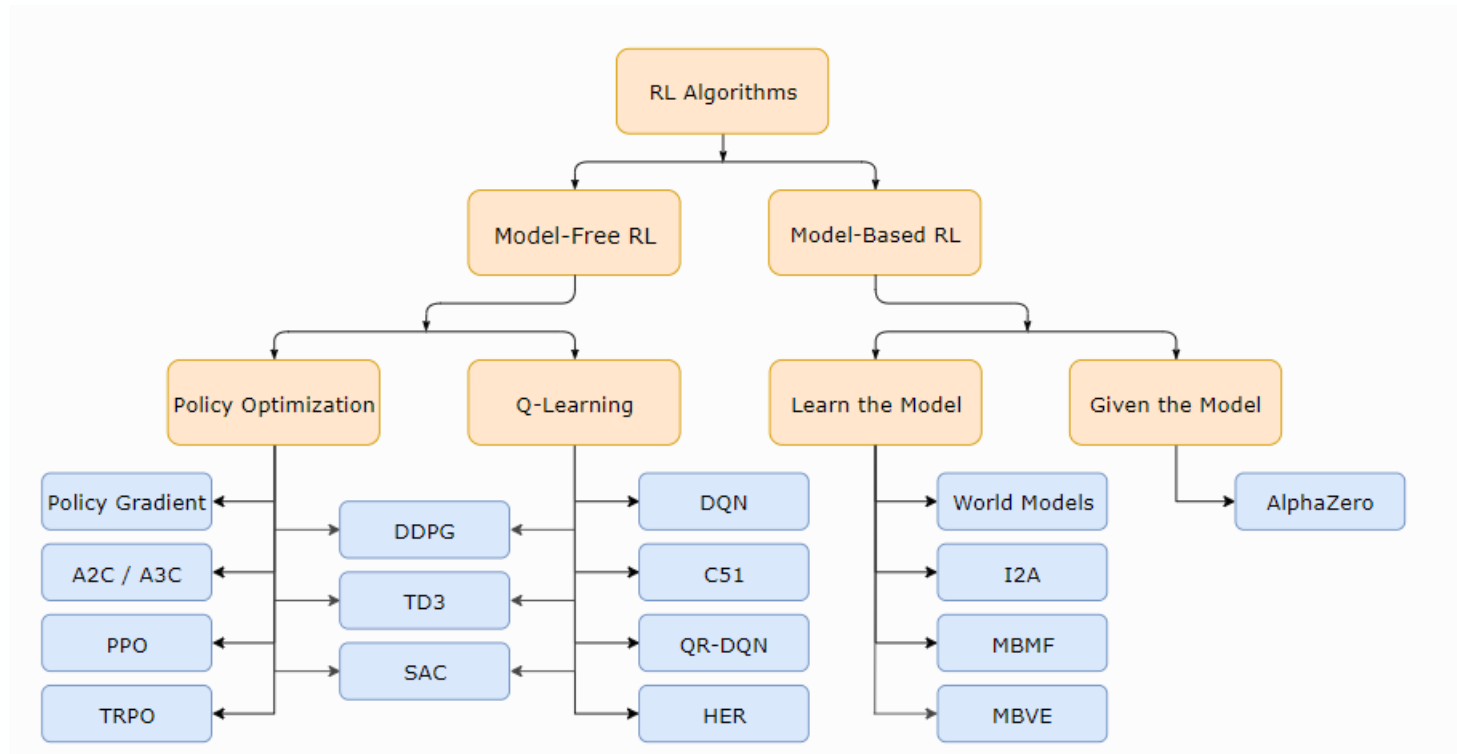
End For

End For

Agenda

- Why Deep Reinforcement Learning?
- DQN Algorithm
- Taxonomy of Deep Reinforcement Learning Algorithms
- Gymnasium Environments
- DQN Code Demo
- Policy Gradient algorithms

Taxonomy of Deep Reinforcement Learning Algorithms



Retrieved from [OpenAI spinning up blog](#)

Agenda

- Why Deep Reinforcement Learning?
- DQN Algorithm
- Taxonomy of Deep Reinforcement Learning Algorithms
- Gymnasium Environments
- DQN Code Demo
- Policy Gradient algorithms

Gymnasium Library

- Gymnasium is a python library that offers multiple simulation environments to test reinforcement learning algorithms in.
- Import the library:
`import gymnasium as gym`
- Create an environment:
`env = gym.make(env_id)`
- Reset the environment:
`observation, info = env.reset()`
- Step the environment with a valid action
`observation, reward, terminated, truncated, info = env.step(action)`

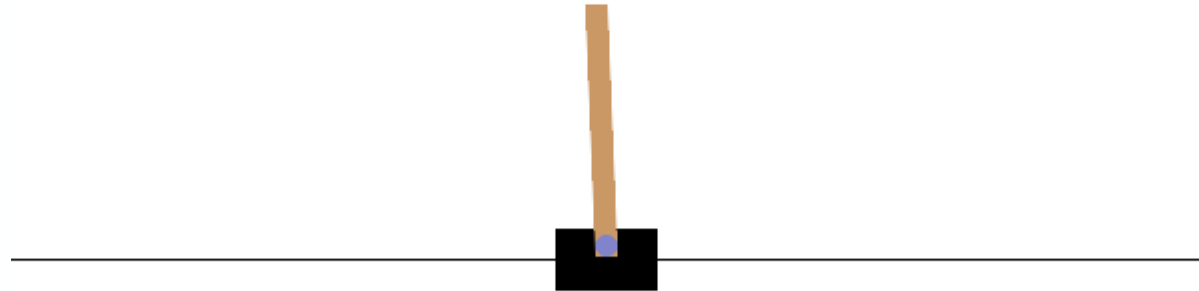
Gymnasium Library

- Get the observation space
env.observation_space
- Get the action space
env.action_space

Agenda

- Why Deep Reinforcement Learning?
- DQN Algorithm
- Taxonomy of Deep Reinforcement Learning Algorithms
- Gymnasium Environments
- DQN Code Demo
- Policy Gradient algorithms

Cart Pole Environment



Agenda

- Why Deep Reinforcement Learning?
- DQN Algorithm
- Taxonomy of Deep Reinforcement Learning Algorithms
- Gymnasium Environments
- DQN Code Demo
- Policy Gradient algorithms

Policy Gradient Algorithms

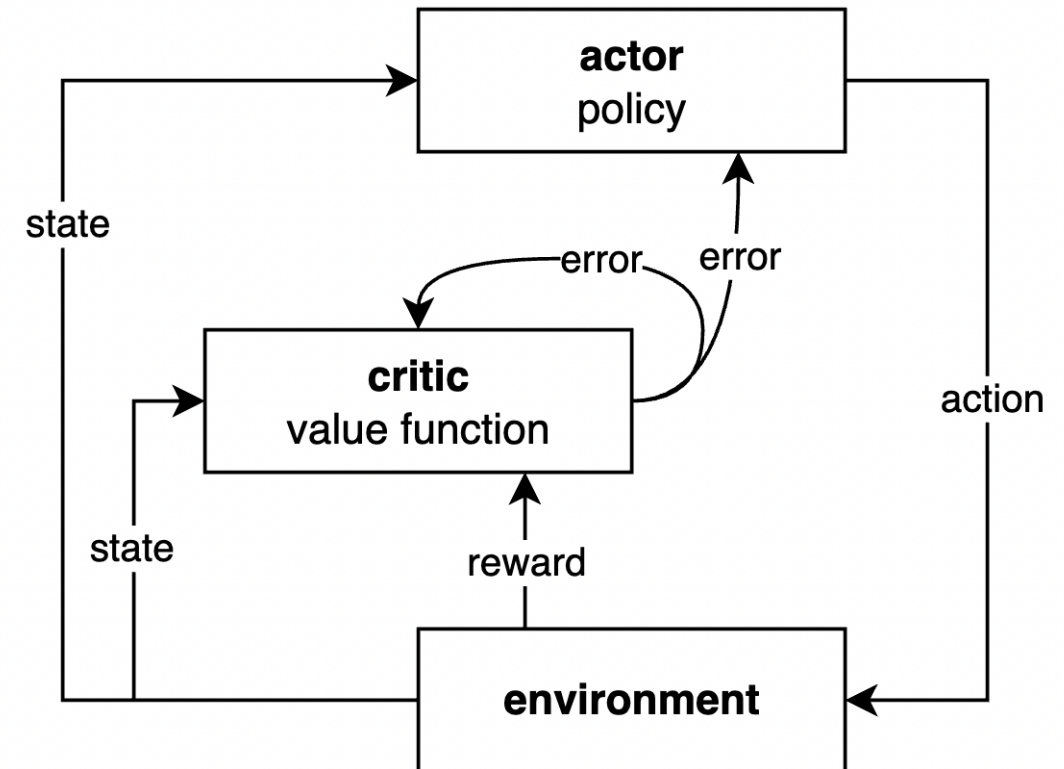
- A policy network π_ϕ parameterized by ϕ is trained to maximize $V^{\pi_\phi}(s_0)$.
- For a deterministic policy $\pi_\phi: \mathcal{S} \rightarrow \mathcal{A}$, the gradient of the loss function used to train the policy network is $\nabla_\phi \mathcal{L} = -\mathbb{E}_{s \sim \rho} \left[\nabla_a Q_\theta(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s) \right]$.
- Deep deterministic policy gradient is an extension to DQN that uses the above loss function to train a policy with continuous actions.

Policy Gradient Algorithms

- For a stochastic policy $\pi_\phi: \mathcal{S} \rightarrow \Delta(\mathcal{A})$, the gradient of the loss function used to train the policy network is $\nabla_\phi \mathcal{L} = -\mathbb{E}_{a \sim \pi, s \sim \rho} [\nabla_a Q_\theta(s, a) \nabla_\phi \log \pi_\phi(a|s)]$.
- Variants of this loss function is used in on policy methods such as Vanilla Policy Gradient (VPG), Proximal Policy Optimization (PPO), and Advantage Actor Critic (A2C).
- The network estimating the value function, or the Q-function is often referred to as the critic, and the policy network is referred to as the actor.

Key Takeaways

- Neural networks are used as function approximators in reinforcement learning to estimate the value function and Q-function and to train a parameterized policy.
- DQN is an extension to Q-learning that utilizes a neural network as an approximator to the Q-function.
- In actor-critic methods, a critic network is trained to estimate the state value function, or the state-action value function and an actor network is trained to find the optimal policy.



Thank You

